

The University of Hong Kong

Faculty of Engineering
Department of Computer Science

COMP3322B Modern Technologies on World Wide Web

Date: May 30, 2020

Time: 2:30 pm - 5:30 pm

Instructions

- This is an open-notes examination. Candidates are permitted to bring to the examination hardcopies of all lecture slides and workshop worksheets, on which handwritten notes are allowed. If students have difficulty in accessing printing facilities, students are allowed to browse/read the materials on their computers.
- Candidates are not allowed to search the Internet for answers during the examination. However, they can use the built-in developer tools in the browsers to assist their coding.
- Setting of the webcam: Set your webcam to capture your faces & eyes and the immediate environment.
- Total of 100 points. This exam consists of 5 questions. Candidates are required to answer all questions.
- If you are not clear about what a question is asking, be sure to write down any assumptions you have made in answering the question.
- To answer the questions, you can type your answers with a computer using Microsoft Word or a text editor. When inserting program code to a Word document, insert the code fragment as text; don't insert the code fragment as image.
- Don't write your answers on paper as we shall screen all submitted answer scripts with plagiarism detection software.
- Put your University Student Number at the beginning of the answer script, followed by the Course code of this examination. Enter the name and type of your calculator, if any, after the Course code.
- Before submitting the answer script to the OLEX system, please make sure that:
 - The answer script is either in Word or PDF format.
 - It includes all your answers and the answers are arranged in the question order.
 - All program code fragments appear as text content (not image content) in the file.

Question 1. Short-answer Questions (25%)

- a) (4%) HTTP is said to be a stateless protocol. Name two enhancements or extensions included in HTTP which make it no longer a true stateless protocol. In one to two sentences, explain how the enhancements/extensions introduce the concept of states to HTTP.
- b) (3%) What is meant by “shorthand property” in CSS? Give one example to illustrate this.
- c) (3%) What is meant by “CSS breakpoint”?
- d) (5%) As the tutor of a course, you are designing an API for accessing the course’s student database. Your API should support the CRUD operations in accessing an individual student record and the list operation to get the full list of student records. Design the API based on the concepts you have learned from the course.

<u>Operation</u>	<u>HTTP method</u>	<u>Path</u>
1. list	_____	_____
2. C	_____	_____
3. R	_____	_____
4. U	_____	_____
5. D	_____	_____

- e) (5%) Consider the following JavaScript code fragment.

```
function sleep( ms ) {  
    return new Promise(resolve => setTimeout(resolve, ms));  
}  
  
console.log("Going to sleep for 3 seconds");  
  
sleep(3000).then(() => console.log("Delay 3 seconds"));  
  
console.log("Have been waiting for 3 seconds");  
  
sleep(1000).then(() => console.log("Delay 1 seconds"));  
  
console.log("Have been waiting for 4 seconds");
```

What would the output be? Justify your answer briefly.

- f) (5%) In setting up the AJAX communication, the 3rd argument to the open function on the XMLHttpRequest object is a Boolean value representing whether the request will be made synchronously (false) or asynchronously (true). Explain what synchronous and asynchronous AJAX calls are. Give an example scenario where using synchronous AJAX call would be appropriate.

Question 2. HTML5 (15%)

Write a Drag-and-drop program with the following features.

1. To **copy** an image, the user drags the image into the <div> element block.
2. An image can be copied into the <div> block multiple times.
3. To **move** an image, the user presses the 'Ctrl' key while dragging the image into the <div> block.
4. Once an image is placed inside the <div> block, it cannot be dragged out of the <div> block.
5. Users cannot create new image object by dragging an image inside the <div> block to another location.
6. The only way to remove an image from the <div> block is to drag-and-drop the image to the trash bin.
7. The trash bin is appeared in the bottom-right of the <div> block.
8. When an image object is being dragged over the trash bin, the graphics of the trash bin changes

from "trashlit.png"  to "trash.png" . When no image object is over the bin, i.e., after the drop or moving the object away from the bin, the graphics of the bin changes back to "trashlit.png".

9. The trash only accepts image objects coming from the <div> block to drop into the bin. When users drag an image from outside of the <div> block to the bin, the bin would not accept the 'drop'.

Below is the HTML framework of the program. Complete the program by adding necessary **JavaScript code** and **CSS styling**. You can use **either jQuery or native JavaScript** to implement the program.

```
<!DOCTYPE HTML>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Drag and Drop</title>
    <style>
      #dest {
        background: lightCyan;
        border: 1px solid #444;
        width: 640px;
        height: 480px;
        padding: 10px;
      }
    </style>
  </head>
  <body>
    <div id='dest'>
      <img id='trash' src='trashlit.png'>
    </div>
    Dragging the image to copy it into the above div element.<br>
    To move, hold the 'Ctrl' key while dragging.<br><br>

    <img id='source1' src='cube.png'>
    <img id='source2' src='Dice2.png'>
    <img id='source3' src='ball.png'>

    <script>
      //Question 2 - Implement the Drag-and-drop program
    </script>
  </body>
</html>
```

//Question 2 - Add necessary styling rules

//Question 2 - Implement the Drag-and-drop program

Below are some screenshots of the program.

Figure 1 When the program starts.



Figure 2 After some copy and move actions.

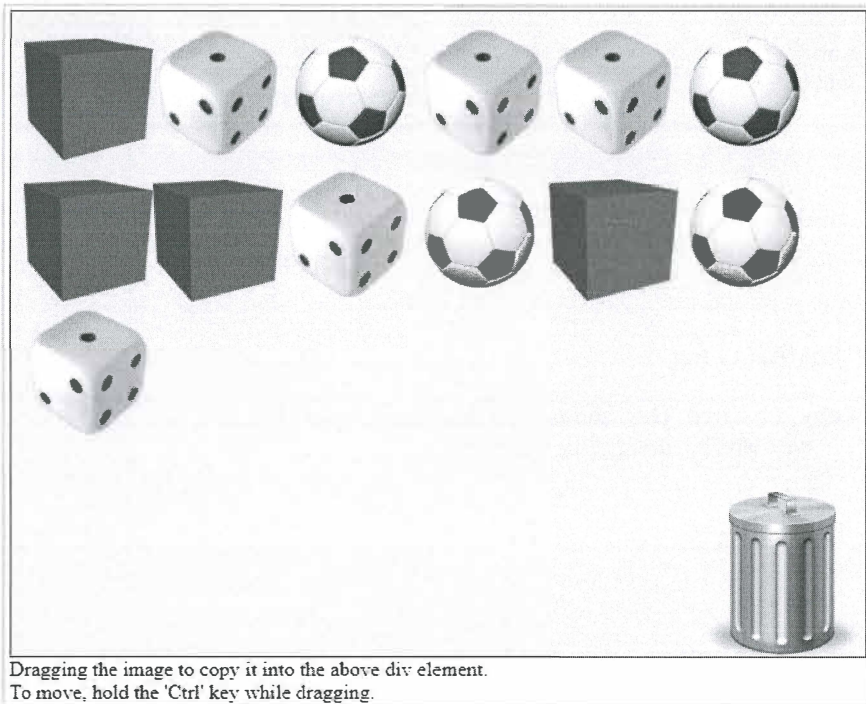
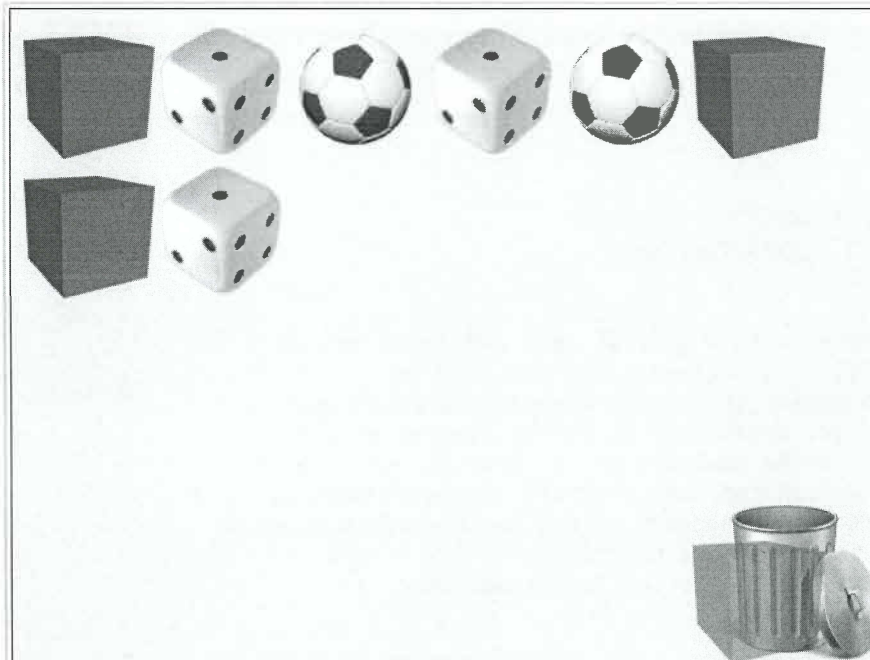


Figure 3 When moving an image object over the trash bin



Dragging the image to copy it into the above div element.
To move, hold the 'Ctrl' key while dragging.

Question 3. jQuery and JavaScript (22%)

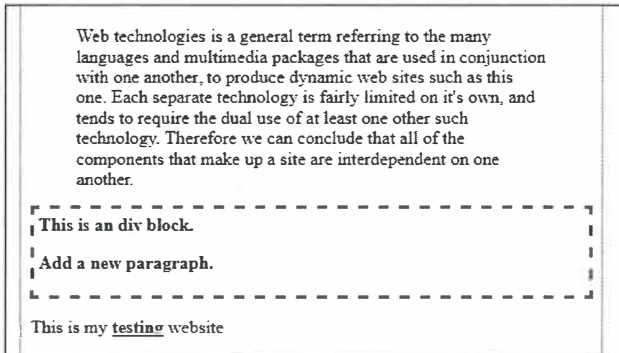
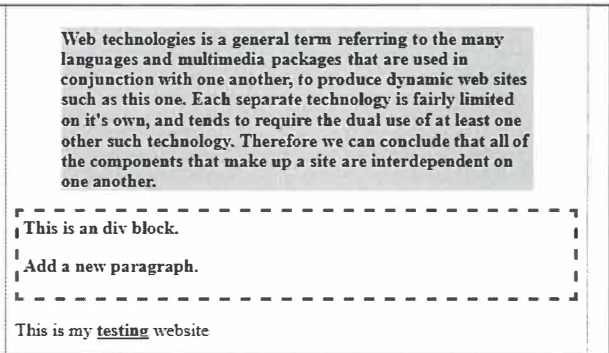
- a) (14%) Below is the jQuery code that implements a simple dynamic web page. Change the code fragment by replacing all jQuery code with the Vanilla JavaScript code that implements the same functionality.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Exam Question</title>
    <script src='jquery-3.3.1.js'></script>
  </head>
  <body>
    <blockquote>Web technologies is a general term referring to the
    many languages and multimedia packages that are used in
    conjunction with one another, to produce dynamic web sites such
    as this one. Each separate technology is fairly limited on it's
    own, and tends to require the dual use of at least one other such
    technology. Therefore we can conclude that all of the components
    that make up a site are interdependent on one another.</blockquote>
    <div id='advert'>This is an div block.</div>
    <p>This is my <span class='new'>testing</span> website</p>
    <script>

      $('blockquote').css('background', 'azure');
      $('.new').css('text-decoration', 'underline');
      $('#advert, .new').css('font-weight', 'bold');
      $('#advert').css('border', '3px dashed red');
      $('#advert').css({'border': '3px dashed red',
        'padding': '4px'});
      $('blockquote').on( {
        'mouseover': function (e) {
          $(this).css({'background': 'lightgray',
            'font-weight': 'bold'});
        },
        'mouseout': function (e) {
          $(this).css({'background': 'azure',
            'font-weight': 'normal'});
        }
      } );
      let $ptext = $('<p></p>').text("Add a new paragraph.");
      $('#advert').append($ptext);

    </script>
  </body>
</html>
```

Here are snapshots of the web page with and without the mouse over the <blockquote> block.

Without mouse over	With mouse over
	

b) (8%) Using the JQuery library to implement two functions – show() and remove() – that work on the following HTML file.

show() – when the mouse enters the <div> with id 'focus', the function:

- adds the image in the file dia.png into the <div> with id 'toshow';
- changes the content of the 'focus' div to "Click me to remove the image";
- removes the onmouseover event and replaces it by the onclick event and adds the remove() function as the event handler.

remove() – this function returns the 'focus' div block back to its previous state. When the user clicks on the 'focus' div block, the function:

- removes the image from the 'toshow' div block;
- changes the content of the 'focus' div to "Mouse over me please";
- removes the onclick event and replaces it by the onmouseover event.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Dynamic load</title>
    <script src='jquery-3.3.1.js'></script>
    <style>
      #focus {
        width: 50%;
        height: 5rem;
        background-color: lightgray;
        text-align: center;
      }
    </style>
  </head>
  <body>
    <div id="focus" onmouseover="show()">
      Mouse over me please
    </div>
    <div id="toshow"></div>
    <script>
      function show() {
        //Question 3b – write the code for the show() function
      }
      function remove() {
        //Question 3b – write the code for the remove() function
      }
    </script>
  </body>
</html>
```


Question 4. Express and MongoDB (20%)

Write an express.js program called **index.js** that pulls data from the MongoDB server in response to the GET request to the route **'/viewmark.html'**. Assume the MongoDB server is running on localhost with the default port number 27017. The name of the database is **midterm**, and the name of the collection is **marks**. Each document consists of 8 fields, which are: **_id**, **name**, **number**, **q1**, **q2**, **q3**, **q4**, and **q5**.

The result of the program is to retrieve all midterm scores from the database and generate an HTML table to present the midterm results. To do that, the index.js program retrieves the data from the database and passes it to the **Pug template engine** to render the viewmark.html page.

You only need to apply the following styling to the html table:

border-collapse: collapse;

border: 1px solid black;

and use the **<h1>** tag to enclose the header **'Midterm Quiz Scores'** .

Notice that you have to compute the total score for each student as well as the average score of the class, which is shown in the last column of the last row.

Below diagrams show the set of data stored in the marks collection and the sample output of the program.

```
> db.marks.find()
{ "_id" : ObjectId("5bfa984d6ccdbbf0ec6ed659"), "name" : "Mary Lam", "number" :
"3015000123", "q1" : 12, "q2" : 10, "q3" : 15, "q4" : 17, "q5" : 15 }
{ "_id" : ObjectId("5bfa984d6ccdbbf0ec6ed65a"), "name" : "Peter Cheng",
"number" : "3015000456", "q1" : 13, "q2" : 16, "q3" : 16, "q4" : 15, "q5" : 17
}
{ "_id" : ObjectId("5bfa984d6ccdbbf0ec6ed65b"), "name" : "Henry Wong", "number"
: "3015000789", "q1" : 10, "q2" : 10, "q3" : 10, "q4" : 10, "q5" : 16 }
{ "_id" : ObjectId("5bfa984d6ccdbbf0ec6ed65c"), "name" : "Angela Lee", "number"
: "3015000321", "q1" : 16, "q2" : 17, "q3" : 18, "q4" : 18, "q5" : 20 }
{ "_id" : ObjectId("5bfa984d6ccdbbf0ec6ed65d"), "name" : "Katherine Chu",
"number" : "3015000987", "q1" : 15, "q2" : 15, "q3" : 15, "q4" : 15, "q5" : 15
}
```

Midterm result

localhost:8000/viewmark.html

Midterm Quiz Scores

Student Name	Student Number	Q1	Q2	Q3	Q4	Q5	Total
Mary Lam	3015000123	12	10	15	17	15	69
Peter Cheng	3015000456	13	16	16	15	17	77
Henry Wong	3015000789	10	10	10	10	16	56
Angela Lee	3015000321	16	17	18	18	20	89
Katherine Chu	3015000987	15	15	15	15	15	75
							73.2

- a) (10%) Complete the index.js program. You must make use of the viewmark.pug template to render the final output of the viewmark.html.

index.js

```
const express = require('express')
const app = express();

app.set("view engine", "pug");
app.set("views", "views");

//Question 4a – complete the program by implementing:

//(1) connect to MongoDB

//(2) Set the Schema and the model

//(3) Handle the route '/viewmark.html'
app.get('/viewmark.html', (req, res) => {

});

app.listen(8000, () => {
  console.log('Example app listening on port 8000!')
});
```

- b) (10%) Complete the viewmark.pug template file.

viewmark.pug

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Midterm result</title>

  //Question 4b – complete the viewmark.pug file
```

Question 5. React (18%)

a) Consider the following React.js app.

```
import React from 'react';
import ReactDOM from 'react-dom';

class App extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      value: this.props.input,
      message: 'click-state 0'
    }
  }

  onClick() {
    this.setState({
      value: this.state.value + 1
    });
    this.setState({
      message: `click-state ${this.state.value}`
    });
  }

  render(){
    return(
      <div>
        <div>current state = {this.state.value} ->
          {this.state.message}
        </div>
        <button onClick={this.onClick.bind(this)}>Please click</button>
      </div>
    );
  }
}

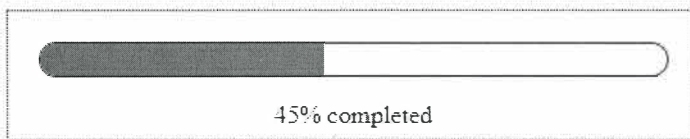
ReactDOM.render(<App input={23} />, document.getElementById("root"));
```

- i. (3%) Draw the output of this React.js app when it is initially loaded by the browser.
- ii. (5%) What is the output of this React.js app when the user clicks on the button 3 times? Briefly explain how this React.js app works to render this result.

- b) (10%) Design a React component that renders a progress bar for visualizing the progression of an event or operation. The progress bar component is composed of the following structure and styling.

```
<div id='container'>
  <div id='bar'>
    <div id='fill'></div>
  </div>
  <p id='text'></p>
</div>
```

The container <div> is for wrapping the components of the progress bar as a single unit. The inner <div>s are for animating the progress of the event/operation. The text <p> is for showing the progress of the operation in textual format. When being displayed, it would look as follows when the status of the progression is at 45 percent:



Implement the progressBar function which accepts an input from the system and passes the input to the progress bar react component to dynamically render the progress of the operation.

```
#container {
  width: 440px;
  border: 1px dotted red;
}
#bar {
  height: 20px;
  width: 400px;
  border-radius: 30px;
  border: 1px solid blue;
  overflow: hidden;
  position: relative;
  left: 20px;
  top: 20px;
}
#fill {
  background: green;
  height: 100%;
  border-radius: initial;
}
#text {
  width: 400px;
  padding-bottom: 10px;
  text-align: center;
  position: relative;
  left: 20px;
  top: 20px;
}
```

```
import React from 'react';
import ReactDOM from 'react-dom';
```

```
function progressBar(percent) {
```

```
  //Question 5b – implement the progress bar react component
```

```
}
```

```
//simulate the feeding of progress data from the system
const steps = [0, 5, 15, 25, 38, 45, 53, 62, 75, 88, 96, 100];
var indx = 0;
```

```
function tick() {
  if (indx === steps.length)
    clearInterval(setTick);
  else
    progressBar(steps[indx++]);
}
```

```
var setTick = setInterval(() => tick(), 1000); //Periodically update
```

*** END OF PAPER ***